

Lab 9: Spring Boot – Transaction

นายพีรพล แก้วเจริญสันติสุข 673380287-8 sec 1

1. การสร้าง Account

Request: POST /accounts

The screenshot shows a REST client interface with a sidebar on the left containing a 'COLLECTIONS' list. The main area displays a POST request to 'http://localhost:8080/accounts'. The request body is a JSON object:

```
{  "accountNumber": "1234567890",  "ownerName": "พีรพล แก้วเจริญสันติสุข",  "balance": 0}
```

. The response status is '200 OK' with a response body:

```
{  "id": 1,  "accountNumber": "1234567890",  "ownerName": "พีรพล แก้วเจริญสันติสุข",  "balance": 0.0}
```

. The bottom status bar indicates 'Ready to go!'.

อธิบาย:

ระบบสร้างบัญชีใหม่สำเร็จ ได้ Status 200 OK และ Response ส่งค่า id: 1 กลับมาพร้อม balance เริ่มต้นเป็น 0.0 ตามข้อมูลที่ส่งไป แสดงว่า endpoint POST /accounts ทำงานถูกต้อง

2. การฝากเงินสำเร็จ

Request: POST /accounts/{id}/deposit

The screenshot shows the Postman interface for a REST client. On the left, the 'COLLECTIONS' sidebar lists 'Lab9' with several endpoints. The 'POST Deposit' endpoint is selected. The main panel shows the request details for 'POST' to 'http://localhost:8080/accounts/1/deposit'. The 'Body' tab is active, displaying a raw JSON payload:

```
{  "amount": 1000}
```

. The bottom panel shows the response status as '200 OK' with a response time of 81 ms and a size of 196 B. The response body is a JSON object:

```
{  "message": "Deposit successful"}
```

. The status bar at the bottom indicates 'Ready to go!'.

อธิบาย:

ส่งค่าขอฝากเงินจำนวน 1000 ไปยัง Account id=1 ได้ Status 200 OK และข้อความ "Deposit successful" กลับมา แสดงว่าระบบประมวลผลการฝากเงินสำเร็จ

3. ตรวจสอบ Account

Request: GET /accounts/{id}

The screenshot shows the Postman interface with a GET request to `http://localhost:8080/accounts/1` successfully executed. The response is a 200 OK status with a response time of 8 ms and a body size of 300 B. The response body is displayed in JSON format, showing account details for id=1.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

```
{
  "id": 1,
  "accountNumber": "1234567890",
  "ownerName": "พิรพชา แก้วเจริญสันติสุข",
  "balance": 1000.0
}
```

อธิบาย:

เมื่อดึงข้อมูล Account id=1 กลับมาดูอีกครั้ง พบว่า balance เปลี่ยนจาก 0 เป็น 1000.0 ตรงกับจำนวนเงินที่ฝากไป ยืนยันว่ายอดเงินถูกอัปเดตในฐานข้อมูลจริง

4. ตรวจสอบ DepositTransaction

pgAdmin: `SELECT * FROM deposit_transaction;`

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure for PostgreSQL 18, with the 'lab9' database selected. The main pane displays a SQL query: `SELECT * FROM deposit_transaction;`. Below the query, the Data Output tab shows the results of the query. The results are displayed in a table with three columns: `id` (PK, bigint), `amount` (double precision), and `account_id` (bigint). The table contains one row with the values: `1`, `1`, and `1000`.

id	amount	account_id
1	1	1000

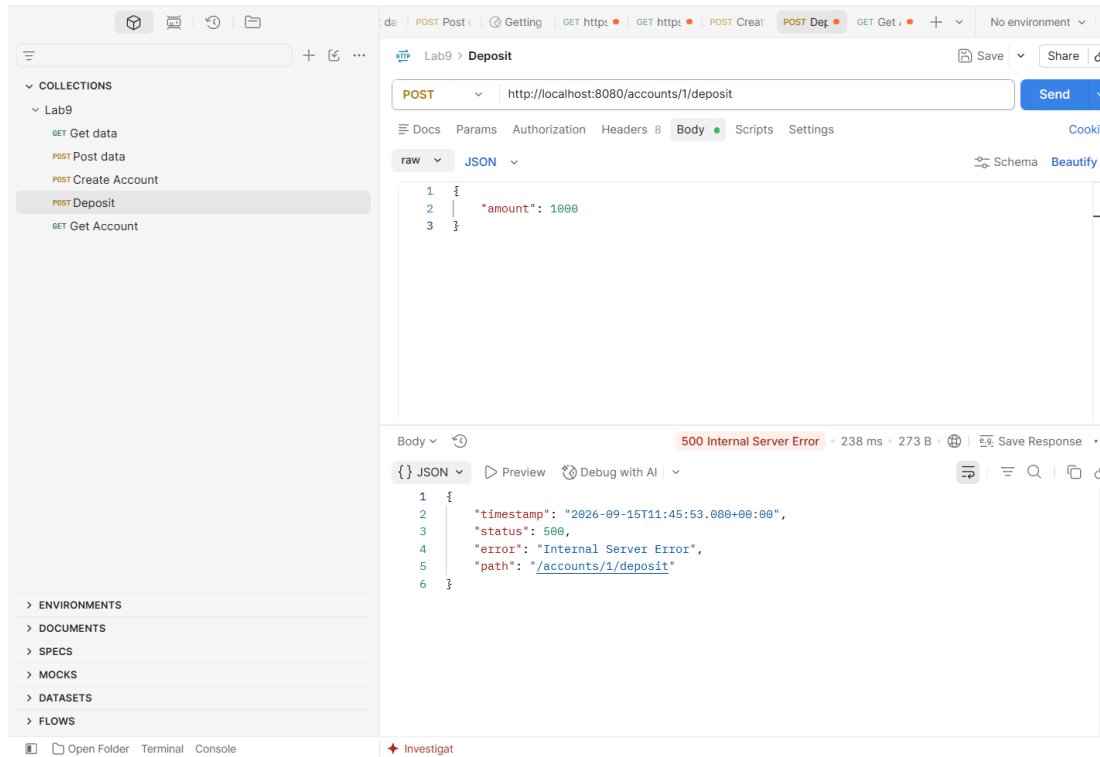
Total rows: 1 Query complete 00:00:00.220

อธิบาย:

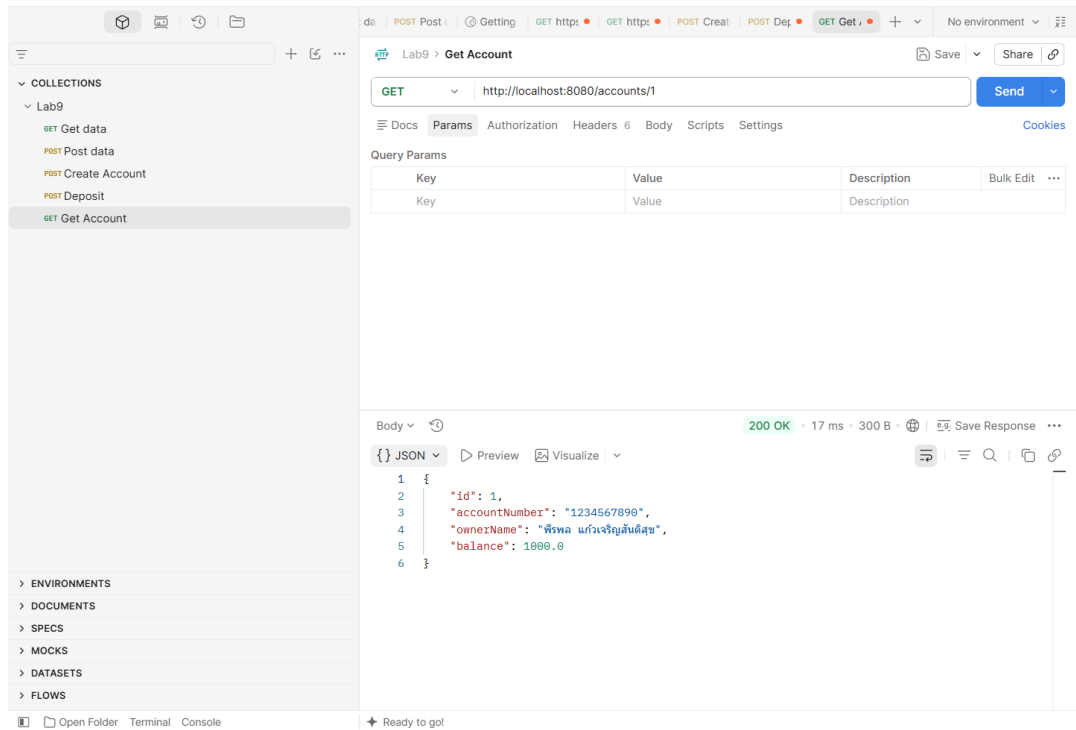
จากการรัน SQL ตรวจสอบตาราง `deposit_transaction` พบ 1 แถว (`amount=1000`, `account_id=1`) แสดงว่านอกจากยอดเงินจะถูกอัปเดตแล้ว ระบบยังบันทึกประวัติการฝากเงินลงตารางแยกได้ถูกต้อง

5. ทดลอง Rollback

Postman แสดง Status 500 หลังเพิ่ม throw new RuntimeException



ตรวจสอบว่าข้อมูลถูกย้อนกลับ (balance ไม่เพิ่ม + deposit_transaction ไม่มีแถวใหม่)



The screenshot shows the PostgreSQL Enterprise Studio interface. On the left, the Object Explorer displays the database structure for PostgreSQL 18, with the 'lab9' database selected. The main query editor shows a SQL query: `SELECT * FROM deposit_transaction;`. Below the query editor, the Data Output pane displays the results of the query in a table format. The table has three columns: 'id' (bigint, PK), 'amount' (double precision), and 'account_id' (bigint). The results show a single row with the values 1, 1000, and 1 respectively. At the bottom right, a green status bar indicates: 'Successfully run. Total query runtime: 83 msec.'

id [PK] bigint	amount double precision	account_id bigint
1	1000	1

อธิบาย:

หลังเพิ่มโค้ด `throw new RuntimeException("Test Rollback")` ต่อท้ายเมธอด `deposit()` แล้วยิงค่าขอฝากเงินเข้า ระบบตอบกลับ Status 500 Internal Server Error ตามคาด เมื่อตรวจสอบข้อมูลอีกครั้งพบว่า balance ยังคงเป็น 1000.0 เท่าเดิม และตาราง `deposit_transaction` ก็ยังมีแค่ 1 แถวเหมือนก่อนทดลองแสดงว่า `@Transactional` ทำงานถูกต้อง คือย้อนกลับ (ROLLBACK) การเปลี่ยนแปลงข้อมูลทั้งหมดเมื่อเกิดข้อผิดพลาดระหว่างการทำงาน แม้โค้ดจะสั่ง `save()` ไปแล้วทั้งสองคำสั่งก่อนเกิด error ก็ตาม

6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional กรณีมี @Transactional

The screenshot displays two applications side-by-side. The top application is a REST client (Postman) showing a POST request to `http://localhost:8080/accounts/1/deposit` with a JSON body `{ "amount": 1000 }`. The response is a `500 Internal Server Error` with a JSON body: `{ "timestamp": "2026-09-15T11:45:53.000+00:00", "status": 500, "error": "Internal Server Error", "path": "/accounts/1/deposit" }`.

The bottom application is a database interface (DBeaver) showing a PostgreSQL 18 database. The query `SELECT * FROM deposit_transaction;` is executed, resulting in a single row:

id [PK] bigint	amount double precision	account_id bigint
1	1000	1

A green status bar at the bottom indicates: `Successfully run. Total query runtime: 83 msec.`

กรณีไม่มี @Transactional

The screenshot shows the Postman interface with a workspace named "My Workspace". On the left, under "COLLECTIONS", there is a collection named "Lab9" containing several API requests. The "POST Deposit" request is selected. The main panel shows the details of this request:

- Method: POST
- URL: `http://localhost:8080/accounts/1/deposit`
- Body (JSON):

```
{  "amount": 1000}
```

The response status is "500 Internal Server Error" with a response time of 13 ms and a size of 273 B. The response body (JSON) is:

```
{  "timestamp": "2026-09-15T11:49:57.559+08:00",  "status": 500,  "error": "Internal Server Error",  "path": "/accounts/1/deposit"}
```

At the bottom, a red message says "Investigate 500 error".

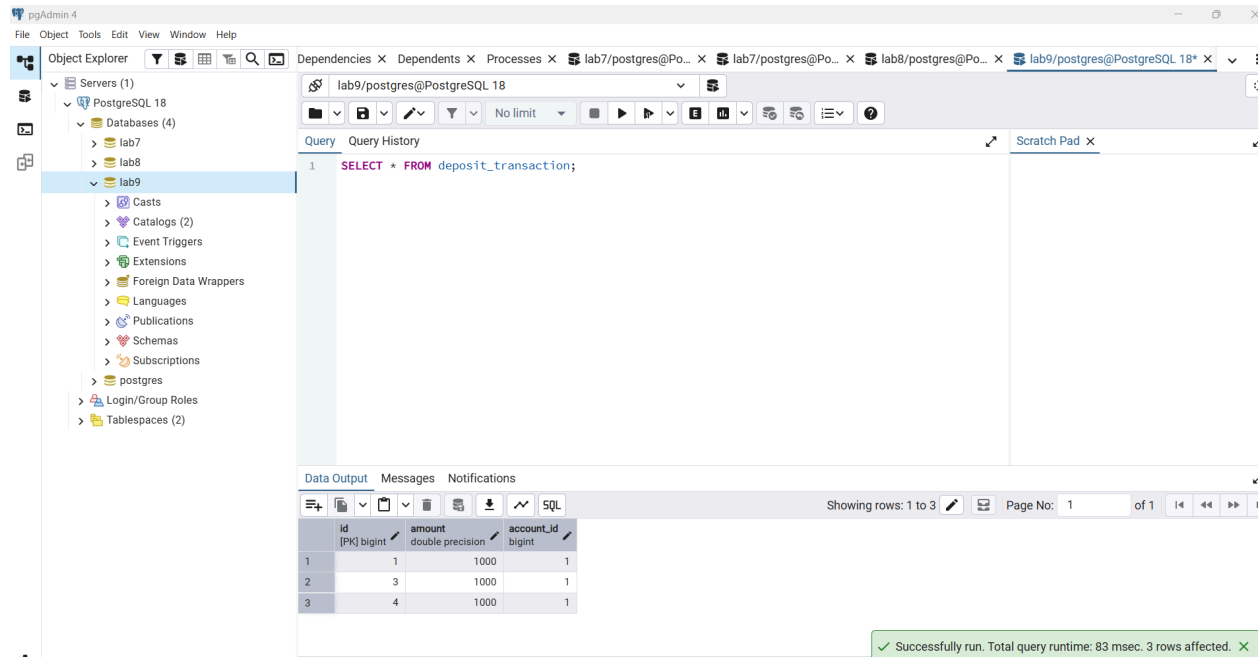
The screenshot shows the Postman interface with the same workspace. The "GET Get Account" request is selected. The main panel shows the details of this request:

- Method: GET
- URL: `http://localhost:8080/accounts/1`
- Query Params: A table with 2 columns: Key, Value, Description, Bulk Edit. The first row has "Key" in the Key column and "Value" in the Value column. The second row has "Value" in the Key column and "Description" in the Value column.

The response status is "200 OK" with a response time of 15 ms and a size of 300 B. The response body (JSON) is:

```
{  "id": 1,  "accountNumber": "1234567890",  "ownerName": "ศิริพร แก้วเจริญสันติสุข",  "balance": 3000.0}
```

At the bottom, a green message says "Ready to go!".



อธิบาย:

ตอนที่ยังมี @Transactional การฝากเงินที่ล้มเหลวจะถูก ROLLBACK ข้อมูลกลับเป็นสภาพเดิมเสมอ balance และตารางประวัติไม่เปลี่ยนแปลง แต่หลังจากลบ @Transactional ออกแล้วทดลองฝากเงินซ้ำ 2 ครั้งโดยที่ยังมี error เกิดขึ้นทุกครั้ง (Status 500 เหมือนเดิม) พบว่า balance กลับเพิ่มขึ้นเป็น 3000.0 และ ตาราง deposit_transaction มีแถวเพิ่มขึ้นเป็น 3 แถว ทั้งที่ค่าขอทุกครั้งจบด้วย error แสดงให้เห็นว่าเมื่อไม่มี @Transactional คำสั่ง save() แต่ละคำสั่งจะถูกบันทึกลงฐานข้อมูลทันทีแยกจากกัน ไม่มีการย้อนกลับแม้จะเกิดข้อผิดพลาดในภายหลัง ทำให้ข้อมูลในระบบไม่สอดคล้องกับความเป็นจริง ต่างจากตอนที่ มี @Transactional อย่างชัดเจน

7. ตอบคำถามท้าย Lab ทั้ง 6 ข้อ

1. @Transactional มีหน้าที่อะไร?

กำหนดขอบเขตของ Transaction ให้ทุกคำสั่งในเมธอดนั้นถูกมองเป็นหน่วยเดียวกัน ถ้าทุกคำสั่งทำงานสำเร็จจะ COMMIT ข้อมูลทั้งหมด แต่ถ้ามีข้อผิดพลาดเกิดขึ้นระหว่างทางจะ ROLLBACK การเปลี่ยนแปลงทั้งหมดกลับไปเหมือนไม่เคยทำอะไรเลย

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เพราะการฝากเงิน 1 ครั้งมีการเปลี่ยนแปลงข้อมูล 2 จุดพร้อมกัน คือ การเพิ่มยอดเงินใน Account และการบันทึกประวัติการฝากใน DepositTransaction ถ้าทำสำเร็จเพียงบางส่วน เช่น เพิ่มยอดเงินแล้วแต่บันทึกประวัติไม่สำเร็จ จะทำให้ข้อมูลในฐานข้อมูลไม่สอดคล้องกัน การใช้ Transaction ช่วยรับประกันว่าทั้งสองขั้นตอนจะสำเร็จไปด้วยกันหรือไม่สำเร็จเลยทั้งคู่

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT คือการยืนยันบันทึกการเปลี่ยนแปลงข้อมูลทั้งหมดที่เกิดขึ้นใน Transaction นั้นลงฐานข้อมูลอย่างถาวร เมื่อทุกคำสั่งทำงานสำเร็จ ส่วน ROLLBACK คือการยกเลิกและย้อนข้อมูลทั้งหมดกลับไปสู่สภาพก่อนเริ่ม Transaction เมื่อมีข้อผิดพลาดเกิดขึ้นระหว่างการทำงาน

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

มีความสัมพันธ์แบบ One-to-Many คือ 1 Account สามารถมี DepositTransaction ได้หลายรายการ โดยฝั่ง DepositTransaction จะเก็บความสัมพันธ์แบบ Many-to-One ผ่านฟิลด์ที่อ้างอิงถึง Account ด้วย Foreign Key ชื่อ account_id ส่วนฝั่ง Account จะไม่มีฟิลด์เก็บลิสต์ของ DepositTransaction เพื่อป้องกันปัญหาการอ่านข้อมูลไม่รู้จักตอนแปลงเป็น JSON

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ข้อมูลจะไม่ถูกบันทึกลงฐานข้อมูลเลยแม้แต่ส่วนที่โค้ดสั่ง save() ไปแล้วก่อนเกิด error เพราะระบบจะ ROLLBACK การเปลี่ยนแปลงทั้งหมดกลับไปเป็นสภาพเดิมก่อนเริ่ม Transaction ทำให้ยอดเงินและประวัติการฝากไม่มีการเปลี่ยนแปลงใด ๆ ปรากฏในฐานข้อมูล

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

เมื่อไม่มี @Transactional คำสั่ง save() แต่ละคำสั่งจะถูก commit ลงฐานข้อมูลทันทีที่คำสั่งแยกจากกัน เมื่อเกิด error ขึ้นภายหลัง ข้อมูลที่บันทึกไปแล้วก่อนหน้านี้ (เช่น ยอดเงินที่เพิ่มไปแล้ว หรือประวัติการฝากที่บันทึกไปแล้ว) จะยังคงอยู่ในฐานข้อมูล ไม่ถูกย้อนกลับ ทำให้ข้อมูลไม่สอดคล้องกัน ต่างจากตอนมี @Transactional ที่ข้อมูลทั้งหมดจะถูกย้อนกลับเป็นสภาพเดิม